

# Digital Voter Information & Management System (VIMS)

*Comprehensive Project Specification & Implementation Blueprint*

|                     |                                                                                                       |
|---------------------|-------------------------------------------------------------------------------------------------------|
| Development Group   | Group B Database Systems Lab, IMSciences                                                              |
| Software Architects | Suhaib Ahmad   Muhammad Rehan Khan                                                                    |
| Academic Degree     | BS Software Engineering                                                                               |
| Deployment Version  | Consolidated Version v2.0 (Updated May 2026)                                                          |
| GitHub Repository   | <a href="https://github.com/SuhaibAhmad0507/digiVIMS">https://github.com/SuhaibAhmad0507/digiVIMS</a> |

## Version Control Log

| Milestone      | Version | Submission Date | Operational Remarks / Changes Documented                                                                 |
|----------------|---------|-----------------|----------------------------------------------------------------------------------------------------------|
| Milestone 1    | v1.0    | April 26, 2025  | Initial EER Diagram & Relational Schema Draft. Revision recommended.                                     |
| Milestone 1    | v1.1    | May 6, 2025     | Added structural description with schema field types and explicit constraints.                           |
| Milestones 2-5 | v2.0    | May 2026        | Consolidated database schema normalization audits, production dataflow, and repository code definitions. |

# Milestone 1: Entity-Relationship Diagram & Schema Description

The Enhanced Entity-Relationship (EER) model details the layout of the digital voting management infrastructure. It maps the direct operational bounds between voters, localized polling stations, household structures (Gharanas), and backend administrative actions. Relationships are strictly controlled at the engine layer via structural foreign key constraints:

- **Voters Core Linking:** Functions as the central node. Relationally binds to Families using family\_id (configured as ON DELETE SET NULL to preserve voter records if a household row is removed) and to PollingStations using station\_id (configured as ON DELETE RESTRICT to secure architectural dependency blocks).
- **Audit Logs Integrity:** Maintains an un-bypassable administrative trail, referencing the initiating Users entity through user\_id (configured as ON DELETE CASCADE to remove associated log footprints cleanly if a user profile is completely purged).

[ Figure 1: Production MySQL Workbench Relational Diagram Schema Layout ]

## Target Architectural Relational Tables

### TABLE: Users

Stores login credentials and security role assignments for system personnel. Supports two access levels: Administrator (full CRUD control) and Viewer (read-only access). Passwords are safe-stored as security-hashed strings.

- user\_id INT PRIMARY KEY AUTO\_INCREMENT
- username VARCHAR(50) NOT NULL UNIQUE
- password\_hash VARCHAR(255) NOT NULL
- role ENUM('Administrator', 'Viewer') NOT NULL

### TABLE: PollingStations

Represents localized geographic voting areas where voters cast structural ballots. Every station handles a unique location code, city designation, and maximum capacity parameters for live load monitoring.

- station\_id INT PRIMARY KEY AUTO\_INCREMENT
- name VARCHAR(100) NOT NULL
- location\_code VARCHAR(20) NOT NULL UNIQUE
- city VARCHAR(50) NOT NULL
- capacity INT NOT NULL

### TABLE: Families

Represents household (Gharana) units grouping multiple individual voters. Records the designated family head and permanent residential address metrics to enable robust household-level extraction pipelines.

- family\_id INT PRIMARY KEY AUTO\_INCREMENT
- head\_name VARCHAR(100) NOT NULL
- permanent\_address TEXT NOT NULL

### TABLE: Voters

The foundational repository tracking individual voter profiles. Enforces absolute uniqueness via CNIC strings, and validates voting eligibility (age >= 18). Mapped dynamically to a Gharana block and Polling Station node.

- voter\_id INT PRIMARY KEY AUTO\_INCREMENT
- cnic CHAR(15) NOT NULL UNIQUE
- full\_name VARCHAR(100) NOT NULL
- age TINYINT NOT NULL CHECK (age >= 18)
- gender ENUM('Male', 'Female', 'Other') NOT NULL
- family\_id INT FOREIGN KEY REFERENCES Families(family\_id) ON DELETE SET NULL
- station\_id INT FOREIGN KEY REFERENCES PollingStations(station\_id) ON DELETE RESTRICT

### TABLE: AuditLogs

Maintains strict, legal-grade system transparency. Intercepts all administrative changes, recording the exact user\_id, operational type (INSERT, UPDATE, DELETE), objective target details, and a server-side timestamp metric.

- log\_id INT PRIMARY KEY AUTO\_INCREMENT
- user\_id INT NOT NULL FOREIGN KEY REFERENCES Users(user\_id) ON DELETE CASCADE
- action VARCHAR(20) NOT NULL (e.g., 'INSERT', 'UPDATE', 'DELETE')
- target\_table VARCHAR(50) NOT NULL
- target\_id INT NOT NULL
- timestamp DATETIME NOT NULL DEFAULT NOW()

## Milestone 2: Normalization Evaluation (1NF → 2NF → 3NF)

To guarantee structural stability and protect performance indices, all core tables have been verified against normal form standard rules. The analysis confirms that the database blueprint designed in Milestone 1 inherently holds a stable 3NF architecture. No modifications were needed.

### Structural Normalization: Users

Attributes: *user\_id* (PK), *username* (UNIQUE), *password\_hash*, *role*

| Form | Relational Evaluation & Analytical Justification                                                                                                                                        | Result Status        |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| 1NF  | Columns store strictly atomic single values. Role is restricted to one ENUM selection; password_hash is a single character string. Zero nested groupings present.                       | No changes required. |
| 2NF  | The primary key is a single-column identifier (user_id). Structural partial dependencies require a composite primary key to exist, meaning 2NF is automatically satisfied.              | No changes required. |
| 3NF  | No non-key column functionally determines another non-key column. Admin privileges do not dictate password values, and username assignments are independent. No transitive links exist. | No changes required. |

### Structural Normalization: PollingStations

Attributes: *station\_id* (PK), *name*, *location\_code* (UNIQUE), *city*, *capacity*

| Form | Relational Evaluation & Analytical Justification                                                                                                                                                                                              | Result Status        |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| 1NF  | Every entity property holds single-valued atomic entries per row. Primary key station_id cleanly identifies each record entry point.                                                                                                          | No changes required. |
| 2NF  | Relying on a single column primary key (station_id) explicitly prevents any partial dependencies from undermining table rows.                                                                                                                 | No changes required. |
| 3NF  | The location_code attribute acts as an alternate candidate key identifying the station block natively (e.g., PSH-001) rather than a broader geographic system mapping. City and capacity properties depend purely and directly on station_id. | No changes required. |

Structural Normalization: Families

Attributes: family\_id (PK), head\_name, permanent\_address

| Form | Relational Evaluation & Analytical Justification                                                                                                                                   | Result Status        |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| 1NF  | Family_id enforces unique row tracking. Attributes head_name and permanent_address contain clear text rows without nested lists.                                                   | No changes required. |
| 2NF  | Single-column key layout handles all row identifiers cleanly. Partial dependency checks are structurally not applicable.                                                           | No changes required. |
| 3NF  | Non-key attributes are fully independent. Two distinct households can share an identical head_name string while residing at different addresses. Transitive properties are absent. | No changes required. |

Structural Normalization: Voters

Attributes: voter\_id (PK), cnic (UNIQUE), full\_name, age, gender, family\_id (FK), station\_id (FK)

| Form | Relational Evaluation & Analytical Justification                                                                                                                                                                            | Result Status        |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| 1NF  | Voter_id acts as primary key. Columns cnic, full_name, age, and gender hold singular atomic values. Foreign keys map to standard reference integers.                                                                        | No changes required. |
| 2NF  | Single-column primary key structure guarantees that all non-key fields describe only the individual voter, confirming compliance with 2NF.                                                                                  | No changes required. |
| 3NF  | CNIC is a unique alternate key identifying a citizen. While CNIC functionally determines full_name, an dependency between two unique candidate keys does not violate 3NF rules. Foreign keys map directly to target tables. | No changes required. |

Structural Normalization: AuditLogs

Attributes: log\_id (PK), user\_id (FK), action, target\_table, target\_id, timestamp

| Form | Relational Evaluation & Analytical Justification                                                                          | Result Status        |
|------|---------------------------------------------------------------------------------------------------------------------------|----------------------|
| 1NF  | Log_id forms the primary key. All action, target_table, and timestamp entries hold atomic data variables with zero multi- | No changes required. |

|     |                                                                                                                                                                                                                                   |                      |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
|     | value arrays.                                                                                                                                                                                                                     |                      |
| 2NF | Single-column log_id primary key leaves no possibility for partial attribute reliance, satisfying all 2NF requirements.                                                                                                           | No changes required. |
| 3NF | Action fields are independent of target_table definitions (e.g., INSERT operations can happen anywhere). Target_id holds situational meaning only when evaluating target_table. Entries are completely free of transitive chains. | No changes required. |

Structural Redundancy Cross-Examination

| Identified Redundancy Risk Context                                   | Architectural Decision & Technical Justification                                                                                                                                   |
|----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| City field replication within the PollingStations table.             | Validated Layout. City maps as a descriptive property of the specific polling station location block, not an independent multi-tier geographic lookup schema.                      |
| Repetitive action strings inside AuditLogs (INSERT, UPDATE, DELETE). | Validated Layout. Using a restricted string layout or native ENUM is ideal. Creating an external lookup entity for 3 static actions introduces unnecessary computational overhead. |
| Matching name attributes between Families and Voters tables.         | Validated Layout. The fields map to completely different entities (family heads vs individual citizens). No overlapping data redundancy exists.                                    |

## Milestone 3: Dataflow Pipeline Description

Information moves smoothly through VIMS across a clear operational pipeline—originating from administrative web management views, passing down into structured relational engines, and displaying back to system views via optimized multi-table JOIN extractions.

### System Entry Points

- **Django Web Admin CRUD Framework:** Secure portal for managing voter records, families (Gharanas), and polling stations. Form submissions execute instant SQL writes (INSERT, UPDATE, DELETE) against the active backend relational database.
- **LOAD DATA INFILE Bulk Seeding Engine:** Optimized data pathway used during initial system setup. Imports large, pre-cleaned legacy CSV files to efficiently seed the relational database with historical registration records.

### Database Execution Order Constraints

Data insertion must closely follow relational dependencies to safely respect engine foreign key controls:

| Step | Target Table    | Depends On                | Operational Pipeline Focus                                                                                                      |
|------|-----------------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| 1    | Users           | None                      | Creates system personnel credentials. Operates independently with zero external foreign key requirements.                       |
| 2    | PollingStations | None                      | Defines the physical boundaries of voting centers. No foreign key constraints exist during creation.                            |
| 3    | Families        | None                      | Instantiates family units (Gharanas) to group household records. Free of external key parameters at initialization.             |
| 4    | Voters          | Families, PollingStations | Links citizens directly to a family cluster (family_id FK) and a polling venue (station_id FK), enforcing relational integrity. |
| 5    | AuditLogs       | Users                     | Automatically tracks administrative writes. Captures the specific user_id to ensure complete operational traceability.          |

Output / System Exit Vectors

Relational data extracts exit via the Django view architecture to serve critical user interface lookups:

- **1. Live Voter Search Interface:** Performs quick lookups by exact CNIC or partial name matching, executing inner joins to bundle a voter's details along with their associated family and polling station information.
- **2. Station-Wise Electoral Rosters:** Generates filtered lists containing all voters assigned to a particular polling venue, providing field staff with official printouts on election day.
- **3. Family Gharana Grouping View:** Queries the central voters table using 'WHERE family\_id = X' to pull and verify all registered members of a specific household cluster.
- **4. Security Audit Ledger:** Combines data from the AuditLogs and Users tables to display a transparent, timestamped record of administrative database operations.

Dataflow Pipeline Matrix Summary

| Pipeline Phase  | Data Source Origin                           | Data Destination                                | Engine Processing Mechanics                                                       |
|-----------------|----------------------------------------------|-------------------------------------------------|-----------------------------------------------------------------------------------|
| Input Phase     | Administrative Web Forms / Raw CSV Data      | Persistent MySQL Relational Tables              | Django ORM Layer / Server-Side LOAD DATA INFILE Optimization                      |
| Storage Phase   | MySQL 8.0 Engine (vims_db Blueprint)         | Fully Normalised Relational Tables              | Robust InnoDB Storage Engine with Native FK Constraints                           |
| Retrieval Phase | Structured SQL SELECT + Complex JOIN Queries | Django View Layer → Standardized HTML Templates | Indexed Database Queries for Guaranteed Sub-Second Lookup Response                |
| Audit Phase     | Any Administrative Database Write Activity   | Dedicated AuditLogs Backend Table               | Automated Security Database Log Handlers Executed on Every Single Write Operation |



## Milestones 4 & 5: GitHub Code Architecture Mapping

|                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Target Deployment URL            | <a href="https://github.com/SuhaibAhmad0507/digiVIMS">https://github.com/SuhaibAhmad0507/digiVIMS</a>                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| Core Architectural Documentation | <ul style="list-style-type: none"><li>• README.md: Contains complete local system deployment walkthroughs and structural architecture overviews.</li><li>• NORMALIZATION.md: Details rigorous proofs confirming the system's 3NF layout across all tables.</li></ul>                                                                                                                                                                                                                                                                          |
| Relational SQL Scripts Directory | <ul style="list-style-type: none"><li>• sql/ddl.sql: Definitive database layout defining schema boundaries, unique indexes, checking criteria, and target constraints.</li><li>• sql/dml.sql: Handles baseline operational data modification scripts, tracking transactional batch additions and secure data row updates.</li><li>• sql/validation.sql: Robust collection of structural data integrity verification tests monitoring target row counts, testing for NULL constraint compliance, and testing foreign key boundaries.</li></ul> |
| Data Testing Infrastructure      | <ul style="list-style-type: none"><li>• data/: Houses raw, clean, synthetic dataset CSV components detailing target variables for immediate test runs.</li><li>• generate_data.py: Executable framework leveraging Python and the Faker utility library to quickly generate massive, clean data volumes to test scaling limits.</li></ul>                                                                                                                                                                                                     |
| Project Git Control Logging      | The repository commit logs mirror the incremental database engineering workflow—tracking early DDL drafts, structural query optimizations, and final validation checks in alignment with project goals.                                                                                                                                                                                                                                                                                                                                       |